

REINFORCEMENT LEARNING

1. INTRODUCTION

Reinforcement learning is a type of ML in which we train agent that learns to behave in an environment by performing actions and observing result of actions.

It involves learning what to do – how to do to map situations to actions, so as to maximise a numerical reward signal.

FEATURES:

- There is no supervisor , only reward signal.
- Involve principal of hit and trial.
- Agent takes next action and manipulates the state according to the feedback of the previous action.
- The agent may receive delayed reward many times.
- The environment that the agent stays in is stochastic, and the agent needs to explore and analyse it to reach the max positive reward.
- Actions affect immediate and subsequent reward as well.

ELEMENTS OF RL:

1. **Reward** : Feedback that the agent receives from the environment to evaluate its action.
2. **Agent** : It is an entity that can perceive / explore its environment and decide to act upon it accordingly.
3. **Environment**: It demonstrates the situation in which an agent is present or surrounded by.
 - i. Fully observable environment: agent directly observes environment state.
 - ii. Partially observable environment: agent indirectly observes the environment. Eg. trading agent
4. **History**: It is the sequence of observations, actions and rewards.

$$H_t = A_1, O_1, K_1, \dots, A_t, O_t, R_t$$

5. **States**: The information used to determine what happens next . It is a situation or state that the environment return after each action that the agent takes.

$$S_t = f(H_t) \text{ [state is a function of history]}$$

- i. Environment state : environment private representation which is not visible to agent.
- ii. Agent state: agent internal representation. It is the information used to pick next action. It is used by RL algorithms.
- iii. Information state: aka Markov state contains all useful information about from history.

MAIN COMPONENTS OF RL AGENT:

- i. **Policy** : It is the strategy applied that agent applies for the next action on the basis of current state. It defines agent behaviour at a given time and maps state to actions.

Deterministic : $a = \pi(s)$

stochastic policy: $\pi(a|s) = P[A = a|S = s]$

- ii. **Value function**: It predicts total amount of reward an agent can expect to accumulate over the future starting from the selected state.

$$V\pi(s) = E\pi [R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots | S_t = s]$$

There are four main functions of note here.

1. The **On-Policy Value Function**, $V^\pi(s)$, which gives the expected return if you start in state s and always act according to policy π :

$$V^\pi(s) = E_{\tau \sim \pi} [R(\tau) | s_0 = s]$$

2. The **On-Policy Action-Value Function**, $Q^\pi(s, a)$, which gives the expected return if you start in state s , take an arbitrary action a (which may not have come from the policy), and then forever after act according to policy π :

$$Q^\pi(s, a) = E_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

3. The **Optimal Value Function**, $V^*(s)$, which gives the expected return if you start in state s and always act according to the *optimal* policy in the environment:

$$V^*(s) = \max_{\pi} E_{\tau \sim \pi} [R(\tau) | s_0 = s]$$

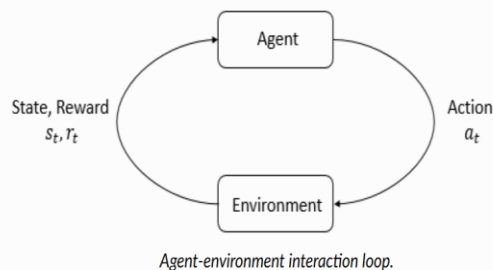
4. The **Optimal Action-Value Function**, $Q^*(s, a)$, which gives the expected return if you start in state s , take an arbitrary action a , and then forever after act according to the *optimal* policy in the environment:

$$Q^*(s, a) = \max_{\pi} E_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

- iii. **Model**: It predicts what environment will do next. It mimics the behaviour of the environment and allow inferences to be made about how the environment will behave. It is a function which predicts state transitions and rewards.

Transition model : predicts the next state

Reward model : predicts next reward



MARKOV DECISION PROCESSES

MDP formally describes an environment for the reinforcement learning where environment is fully observable. Almost all RL problems can be formalized as MDP.

Markov Property: The future is independent of the past given the present.

MDP is a Markov reward process with decisions. It is where all states are Markov. It is represented by (S, A, P, R, γ) where :

S: finite set of states

A: finite set of actions

P: state transition matrix

R: reward function

γ : discount factor

RETURN

Return (usually denoted as G_t) is the total cumulative reward an agent receives from a specific time step t until the end of the episode.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum \gamma^k R_{t+k+1}$$

γ (gamma) is a number between 0 and 1.

A γ closer to 0 makes the agent focus on immediate rewards.

A γ closer to 1 encourages long-term planning. [[1](#), [2](#), [3](#)]

BELLMAN EQUATION

The state value function can again be decomposed into immediate into immediate reward plus discounted value of successor state.

The Bellman equations for the on-policy value functions are

$$V^\pi(s) = \mathbb{E}_{\substack{a \sim \pi \\ s' \sim P}} [r(s, a) + \gamma V^\pi(s')],$$
$$Q^\pi(s, a) = \mathbb{E}_{s' \sim P} \left[r(s, a) + \gamma \mathbb{E}_{a' \sim \pi} [Q^\pi(s', a')] \right],$$

where $s' \sim P$ is shorthand for $s' \sim P(\cdot | s, a)$, indicating that the next state s' is sampled from the environment's transition rules; $a \sim \pi$ is shorthand for $a \sim \pi(\cdot | s)$; and $a' \sim \pi$ is shorthand for $a' \sim \pi(\cdot | s')$.

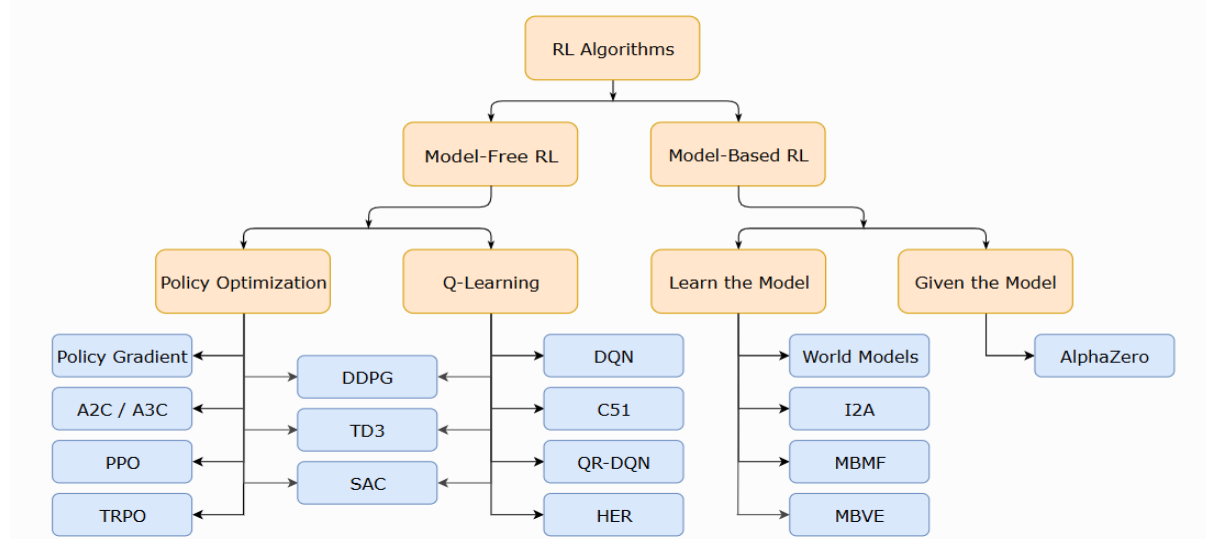
The Bellman equations for the optimal value functions are

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P} [r(s, a) + \gamma V^*(s')],$$
$$Q^*(s, a) = \mathbb{E}_{s' \sim P} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right].$$

The crucial difference between the Bellman equations for the on-policy value functions and the optimal value functions, is the absence or presence of the $\max$ over actions. Its inclusion reflects the fact that whenever the agent gets to choose its action, in order to act optimally, it has to pick whichever action leads to the highest value.

RL ALGORITHMS : FROM CLASSICAL METHODS TO DEEP RL

A Taxonomy of RL Algorithms



MODEL FREE VS MODEL BASED RL

RL algorithms are classified into two categories based on whether the agent has access to a model of the environment: **model-based** and **model-free** methods.

The advantage of having a model, helps agent to plan ahead by simulating different actions and evaluating their consequences before making a decision. By comparing multiple possible outcomes, the agent can choose the most promising action. The knowledge gained through this planning process can then be distilled into a learned policy, allowing faster decision-making during execution.

But for the majority of large problems the agent does not have a policy and has to learn the model from its own experience while taking actions. In such cases, the agent must learn the model from its own experience while interacting with the environment. Since the learned model may be inaccurate, the agent can sometimes develop strategies that perform well in the learned model but fail in the real environment. For this reason, learning an accurate model is challenging, and many applications rely on model-free methods that learn directly from experience without using a model.

Classical RL Classification (Sutton & Barto)

1. Dynamic Programming :

Dynamic programming assumes full knowledge of MDP and is used for planning in an MDP. It uses Bellman equation which expresses the value of a state as immediate reward plus the discounted value of future states.

$$V\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V\pi(s')]$$

Key DP methods:

Policy evaluation- Computes the value function for a given policy.

Policy improvement- Improves the policy using the estimated value function.

Policy iteration- Alternates between policy evaluation and policy improvement.

Value iteration- Combines evaluation and improvement into a single update process.

Advantages – Guaranteed convergence and efficient when model is available.

Disadvantages- Requires complete knowledge of MDP, so not suitable for large-scale problems.

2. Monte Carlo Methods:

MC methods learn from complete episodes of experience and is a model free RL method. Instead of estimating values using model, MC methods wait until an episode ends and then calculate the actual return received.

Types:

First visit MC: Updates a state only the first time it appears in an episode.

Every visit MC: Updates every occurrence of a state.

Advantages- No model required and unbiased estimates.

Disadvantages- Must wait until episode ends and has high variance.

3. Temporal Difference learning:

Temporal difference combines ideas from Monte Carlo methods and DP and is a model free approach. Unlike MC, TD updates estimates before an episode ends.

TD error : $\delta = R_t + 1 + \gamma V(S_{t+1}) - V(S_t)$

TD(0) updation: $V(S_t) \leftarrow V(S_t) + \alpha \delta$

Advantages- Learns online, faster than MC and lower variance.

Disadvantages- Introduces bias through bootstrapping.

- a. **SARSA** : It updates the values using the action actually chosen by the current policy(epsilon greedy), making it aware of the agent's exploration behavior. As a result, SARSA learns policies that account for the risks associated with exploratory

actions and often produces safer behavior in uncertain environments. Here, Learning policy = behavior policy.

Update Rule: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$

Characteristics: On-policy, uses actual action a' , learns safer behaviour and considers exploration actions.

- b. **Q- Learning** : It updates values using the best possible future action. Instead of using the action actually taken, it updates Q-values using the maximum estimated future reward available in the next state. This allows Q-Learning to learn the optimal greedy policy regardless of the exploration strategy being used, often leading to faster convergence toward the optimal solution. Here, Behavior policy $\neq$ learning policy.

Update Rule: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

Characteristics: Off-policy, uses best action $\max_{a'} Q(s', a')$, learns optimal behaviour and more aggressive learning.

[Implementation : SARSA & Q-learning](#)

ON-POLICY VS OFF-POLICY

This difference depends on whether the learning policy and behaviour policy are the same.

On-policy : Agents learns from the actions generated by the same policy it is trying to improve. Eg. SARSA, PPO, A2C

Off-policy : Agents learns about one policy while following another. Eg. Q-learning, DQN etc.

Modern Deep RL Classification

Classical RL methods use lookup tables to store value estimates for state-action pairs. This is effective for small environments, but become impractical when the state or action space is large or continuous. For eg. Autonomous driving cars, robotics and videos games contain millions of possible states, making tabulation representation infeasible.

To overcome this limitation, Deep RL combines RL with deep neural networks. Instead of storing values in tables, neural networks are used to approximate policies and value functions, enabling agents learn complex environments.

Function Approximation -

- **Value Function Approximation:** Value Function Approximation (VFA) uses a function, typically a neural network, to estimate values instead of storing them explicitly.

Approximate state-value: $V(s) \approx v(s, \theta)$

Approximate action-value: $Q(s, a) \approx Q(s, a, \theta)$

Where, θ is parameters (weights) of the approximator

The function takes a state (or state-action pair) as input and predicts its value.

- **Policy function approximation:** Similarly, policy can be represented as: $\pi(a | s; \theta)$. This allows the agent to generalize its learning to previously unseen states.

POLICY OPTIMIZATION:

The goal is to **directly optimize the policy** $\pi\theta(a | s)$, instead of learning values and deriving actions from them, the algorithm learns the policy parameters θ directly to maximize expected return.

- Policy gradient methods : Policy gradient methods update the policy parameters in the direction that increases expected rewards. The goal is to maximize the expected return:

$$J(\theta)$$

Advantages : can learn stochastic policy also and directly optimise the policy.

Limitations: High variance in updates and requires many training samples.

➤ REINFORCE Algorithm:

REINFORCE is the simplest policy gradient algorithm. It updates the policy after observing complete episodes and uses the total return obtained from the episode to adjust policy parameters.

Characteristics

- Monte Carlo based.
- On-policy learning.
- Simple implementation.
- High variance.

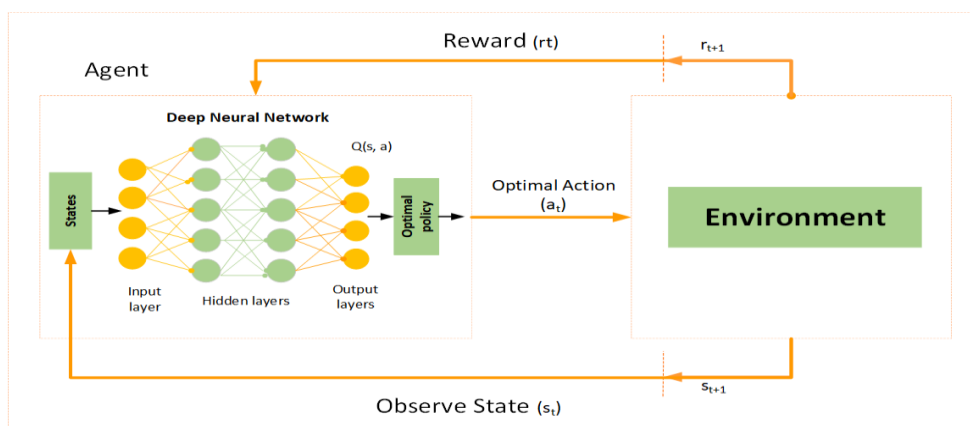
REINFORCE laid the foundation for many modern policy optimization algorithms.

VALUE OPTIMIZATION:

The goal is to **learn value functions** such as $V(s)$ or $Q(s, a)$. The policy is then obtained by selecting actions with highest estimated value.

DQN : DQN extends Q-Learning by using a neural network to approximate the action-value function. Instead of maintaining a Q-table, DQN learns $Q(s, a; \theta)$ where the neural network predicts values for different actions.

DQN Architecture :



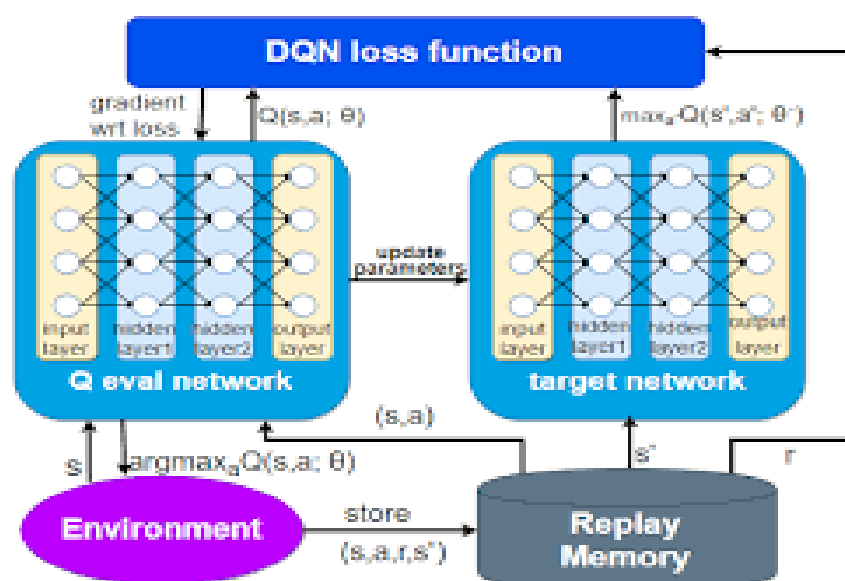
Input layer has the nodes that represent the states and output layer nodes represent actions. The state is taken as input and the Q-value with highest output is given as output.

Challenges in deep RL :

1. **Correlated sample** – training on consecutive sequential states causes samples to be highly correlated. This violates the independent and identically distributed (i.i.d.) assumption of gradient descent, causing the network to overfit to localized state-action trajectories, forget past experiences (catastrophic forgetting), and ultimately diverge or become unstable.
2. **Moving target** - In standard supervised learning, the target labels remain static. In DQN, the loss function measures the difference between your Q-value estimation and the target. Every time you perform a gradient update, the network's weights change slightly, which inherently shifts both the current estimate *and* the target. Chasing a constantly moving target introduces severe divergence, oscillations, and even causes the network to collapse and "forget" previously learned policies.

Solutions to these challenges :

1. **Experience Replay** – It stores the agent's experiences as transition tuples (s, a, r, s') in a large replay buffer. During training, random mini-batches of these stored experiences are sampled and used to update the neural network. By randomly selecting experiences rather than using consecutive samples, experience replay breaks the strong temporal correlations present in sequential data and helps create a more independent and identically distributed (i.i.d.) training set. This improves learning stability, increases data efficiency, and enhances the performance of deep reinforcement learning algorithms such as DQN.
2. **Target Network** - It is a stabilization technique used in Deep Q-Networks (DQN). Instead of using the same neural network to both predict Q-values and generate target values, DQN maintains a separate target network whose parameters are copied periodically from the main Q-network. The target network is used to compute the target Q-values during training, while the main network is updated through gradient descent. Because the target network changes less frequently, the learning targets remain more stable, reducing oscillations and preventing divergence during training.



ACTOR CRITIC METHODS:

Actor-Critic methods are a class of reinforcement learning algorithms that combine the strengths of policy-based and value-based approaches. Instead of relying solely on a policy or a value function, these methods use two separate components: an Actor, which selects actions, and a Critic, which evaluates the quality of those actions.

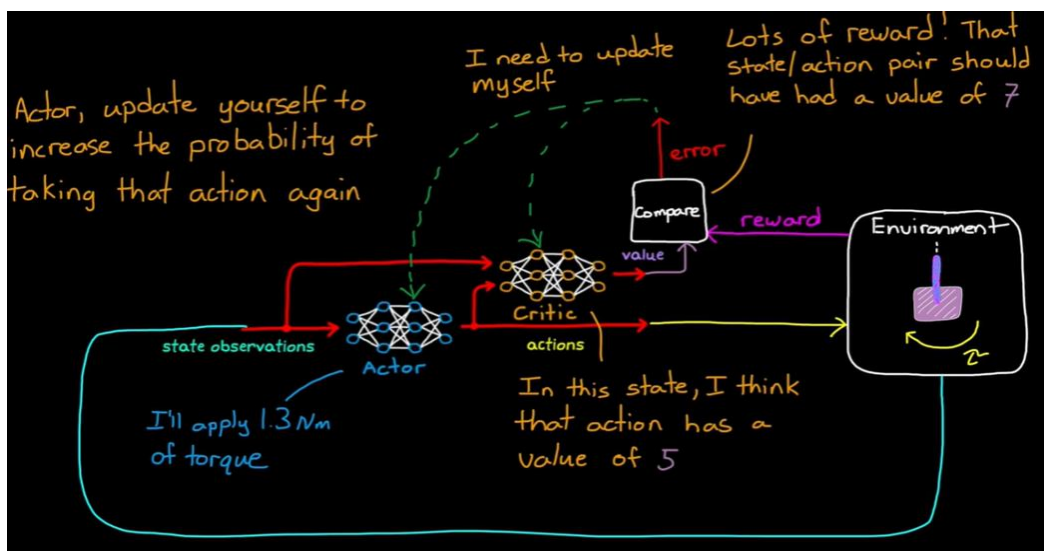
The Actor-Critic framework addresses the limitations of pure policy gradient methods, which often suffer from high variance, and value-based methods, which may struggle with continuous action spaces. By combining both approaches, Actor-Critic methods achieve more stable and efficient learning.

- The **Actor** is responsible for learning and updating the policy. It represents a policy : $\pi(a | s)$ which maps states to actions. Based on the current state, the actor decides which action to take in order to maximize long-term rewards.
 - Select actions.
 - Learn the optimal policy.
 - Update policy parameters using feedback from the critic.
- The **Critic** evaluates the actions selected by the actor. It estimates a value function such as: $V(s)$ or $Q(s, a)$. The critic measures how good a state or state-action pair is and provides feedback that helps the actor improve its policy.
 - Estimate future rewards.
 - Compute learning signals for the actor.
 - Reduce variance in policy updates.
- **Advantage Function** – A commonly used feedback signal is the **Advantage Function**, which measures how much better an action is compared to the average action in a state. $A(s, a) = Q(s, a) - V(s)$

Interpretation

- $A(s, a) > 0$: action is better than expected.
- $A(s, a) < 0$: action is worse than expected.
- $A(s, a) = 0$: action performs as expected.

The actor uses this information to increase the probability of good actions and decrease the probability of poor actions.



Popular Actor-Critic Algorithms

- **A2C (Advantage Actor-Critic)**
A synchronous actor-critic algorithm where multiple agents collect experiences and update a shared model simultaneously.
- **A3C (Asynchronous Advantage Actor-Critic)**
An extension of A2C that uses multiple parallel agents operating asynchronously to improve exploration and training speed.
- **PPO (Proximal Policy Optimization)**
One of the most widely used actor-critic algorithms. PPO improves training stability by limiting the size of policy updates through a clipped objective function.
- **DDPG (Deep Deterministic Policy Gradient)**
An off-policy actor-critic algorithm designed for continuous action spaces. It combines deterministic policies with experience replay and target networks.
- **SAC (Soft Actor-Critic)**
A modern actor-critic algorithm that incorporates entropy regularization to encourage exploration and improve learning stability.

EXPLORATION – EXPLOITATION TRADE-OFF

Exploitation – It refers to choosing the action that currently has the highest estimated value based on the agent's past experience. For example, if an agent has learned that a particular action consistently provides a high reward, it may repeatedly select that action to maximize immediate returns.

Advantages:

- Maximizes short-term rewards.
- Uses existing knowledge efficiently.

Disadvantages:

- May miss better actions that have not been tried enough.
- Can lead to suboptimal solutions.

Exploration – Exploration involves selecting actions that the agent has not tried frequently or whose outcomes are uncertain. By exploring, the agent gathers new information about the environment and may discover actions that yield higher rewards in the long run.

Advantages:

- Helps discover better strategies.
- Improves knowledge of the environment.

Disadvantages:

- May temporarily reduce rewards.
- Can lead to inefficient behavior during learning.

The Trade-off

The challenge is to balance exploration and exploitation.

- Too much **exploitation** may cause the agent to get stuck with a suboptimal policy.
- Too much **exploration** may prevent the agent from fully utilizing the knowledge it has already acquired.

An effective RL agent must explore enough to discover good actions while exploiting enough to maximize rewards.

Common Exploration Strategies

1. ϵ -Greedy Strategy

The most widely used exploration method.

- With probability ϵ , choose a random action (exploration).
- With probability $1 - \epsilon$, choose the best-known action (exploitation).

For example, if: $\epsilon = 0.1$

then:

10% of the time $\rightarrow$ random action.

90% of the time $\rightarrow$ best action.

2. Decaying ϵ -Greedy

Initially, the agent explores heavily.

As learning progresses, ϵ is gradually reduced so that the agent increasingly exploits its learned knowledge.

REFERENCES

- [1] <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [2] <https://spinningup.openai.com/en/latest/>
- [3] <https://huggingface.co/learn/deep-rl-course/unit3/deep-q-algorithm>
- [4] <https://pythongeeks.org/reinforcement-learning-in-machine-learning/>
- [5] <https://davidstarsilver.wordpress.com/teaching/>
- [6] <https://www.youtube.com/watch?v=7cF3VzP5EDl>
- [7] <https://youtube.com/playlist?list=PLzuuYNsE1EZAXYR4FJ75jcJseBmo4KQ9-&si=T08SWt2sT0ChwDRE>